
eateggsAI
eateggsAI-49.34M
v0.1
Technical White Paper
Building a Small Language Model
Under a 4 GB VRAM Constraint
49.34M
Parameters
4 GB
VRAM
From Scratch
with PyTorch
Transformer Block ×6
Token Embedding
Positional Embedding
Layer Norm
Multi-Head Self-Attention
Layer Norm
Feed Forward (GELU)
LM Head
Output (Next Token)
Transformers
Language Models
PyTorch
Machine Learning
Low-VRAM AI
eateggai
eateggsAI
Small Models Big Dreams
Author: Eateggs
Project: eateggsAI-49.34M v0.1
Developer Identity: eateggai
OPEN SOURCE | LEARN | EXPERIMENT | IMPROVE

Abstract

This paper documents eateggsAI-49.34M v0.1, an experimental small language model built from scratch using PyTorch. The supplied project README reports a 49,343,232-parameter GPT-style decoder-only Transformer trained on an NVIDIA GTX 1060 with 4 GB of VRAM. The documented configuration includes six Transformer layers, eight attention heads, a 384-dimensional embedding, a 256-token context length, and a 50,257-token GPT-2 vocabulary. The README reports approximately 12 hours of training, about 2.7 GB VRAM usage, and a final loss of 3.2–3.3. These are author-reported figures; this document does not claim independent reproduction or validation of the results.

1. Project Overview

eateggsAI-49.34M is an educational and experimental small language model (SLM) project. Its central engineering question is whether a compact GPT-style Transformer can be trained on older consumer hardware with a 4 GB GPU-memory limit, without depending on high-memory cloud accelerators.

The project README describes a model implemented from scratch in PyTorch, a multi-domain text preparation pipeline, mixed-precision training, and an inference script for text generation.

2. Goals and Scope

- Implement a decoder-only Transformer and train its parameters using PyTorch.
- Explore a low-VRAM training setup on an NVIDIA GTX 1060 with 4 GB VRAM.
- Prepare text from multiple domains using cleaning, deduplication, tokenization, and fixed-length block packing.
- Save trained weights and run inference.
- Document the configuration and reported outcomes without overstating what the available evidence proves.

3. Model and Hardware Configuration

Item	Configuration reported in README
Project/version	eateggsAI-49.34M v0.1
Parameter count	49,343,232 (approximately 49.34 million)
Architecture	GPT-style decoder-only Transformer
Transformer layers	6
Attention heads	8
Embedding dimension	384
Context length	256 tokens
Vocabulary size	50,257
Dropout	0.1
GPU	NVIDIA GTX 1060
GPU memory	4 GB VRAM
Laptop	Acer i5
Software	Python 3.11, PyTorch, CUDA-supported environment

4. Low-VRAM Training Approach

Mixed precision

The README describes PyTorch automatic mixed precision (AMP), using autocast and gradient scaling. Mixed precision is intended to reduce memory requirements for supported operations. Exact numerical settings and the full training script should be consulted to reproduce the run.

Sequence length and batch size

The documented context length is 256 tokens. The README lists a batch size of 11 as part of its low-VRAM configuration. These values describe the reported setup; their impact should be verified through training logs and repeatable measurements.

Optimization and objective

- AdamW optimizer.
- Cross-entropy loss.
- Mixed precision and gradient scaling.
- CUDA acceleration.
- Saving trained model weights for inference.

5. Dataset Sources and Preparation

The supplied README names Wikipedia, AG News, SQuAD, and PG19 Books as dataset sources. It describes the following preparation stages:

- Multi-domain dataset streaming and balancing.
- Data cleaning and deduplication.
- GPT-2 tokenization.
- Packing token IDs into fixed-length blocks.
- Conversion to PyTorch tensors for training.

The README does not provide dataset versions, exact source proportions, licensing analysis, a complete filtering specification, or train/validation/test split details. Those items should be recorded before claiming the experiment is fully reproducible.

6. Architecture Summary

The README identifies these components in the GPT-style decoder-only model:

- Causal multi-head self-attention.
- GELU activation.
- Residual connections.
- Layer normalization.
- Learned positional embeddings.
- Causal attention masking.
- GPT-2 tokenizer vocabulary of 50,257 tokens.

Figure 1 shows the high-level data and training workflow described by the README. It is a schematic created for this document, not a screenshot of the original implementation.



Figure 1. Schematic of the dataset-to-training workflow described in the README.

7. Training Workflow

The documented pipeline is: dataset collection → cleaning and deduplication → GPT-2 tokenization → fixed-length block creation → Transformer training → loss optimization → model saving.

The README lists scripts for downloading datasets, cleaning data, preparing blocks, training, and inference. The repository URL in the supplied README contains a placeholder username and an older project slug; confirm the current repository address and file paths before publishing setup instructions.

8. Reported Results

Metric	Value reported in README
Training time	Approximately 12 hours
VRAM usage	Approximately 2.7 GB
Final loss	3.2–3.3
Precision	Mixed precision (AMP)
Framework	PyTorch

The README does not specify the exact loss aggregation method, validation loss, evaluation dataset, number of training tokens for this run, random seed, or repeated-run variance. A final training loss alone does not establish general language ability or factual reliability, and should not be compared directly against differently configured experiments.

9. Inference and Demonstrations

The README includes a sample-generation screenshot and gives “Why is the sky blue?” as an example prompt. The original image asset links are listed below; the assets could not be embedded from the source URLs in this generated edition, so the links are retained for reference.

- **Dataset overview image:** https://github.com/user-attachments/assets/c83d65b3-88ef-40c4-aebf-5cf42c572f7e
- **Raw datasets image:** https://github.com/user-attachments/assets/8389f0f8-f378-4592-9796-d99235b84392
- **Sample generation output:** https://github.com/user-attachments/assets/c04ca551-3679-4ac9-856b-925b662e43ab

10. Reproduction Instructions

The README provides the following commands. They should be treated as examples until the repository URL, directory names, and script paths have been checked against the current repository.

```
git clone https://github.com/eateggs0989/eateggsAI-49.34M.git
cd eateggsAI-49.34M
pip install -r requirements.txt

python dataset/download_datasets.py
python dataCleaner.py
python dataset/prepare_blocks.py

python training/train.py
python inference/generate.py
```

The README says the trained weights are saved as gpt_6layer.pt. Confirm that this filename and the listed scripts match the actual project version.

11. Limitations and Recommended Next Steps

This document is based on the supplied README, not an independent audit of the code or training artifacts. The reported hardware and training metrics are therefore presented as project-reported values.

- Add author name or preferred project attribution, date, and repository URL.
- Embed the original dataset and inference screenshots at readable resolution.
- Record exact dependency versions, training configuration, random seeds, dataset versions, and token counts.
- Create a clearly documented validation split and report validation loss in addition to training loss.
- Include representative prompts and unedited model outputs, with the sampling settings used.
- Run repeatable experiments if making claims about the effect of low-VRAM optimizations.
- Add properly formatted references for the Transformer architecture, PyTorch, tokenizer, and datasets.

12. Conclusion

eateggsAI-49.34M v0.1 documents an attempt to train a roughly 49.34-million-parameter GPT-style language model on an NVIDIA GTX 1060 with 4 GB VRAM. The README reports a six-layer architecture, 256-token context, mixed-precision training, approximately 2.7 GB VRAM usage, and a final loss of 3.2–3.3. The project is a useful educational engineering artifact for exploring Transformer implementation and low-memory training. Further evaluation and reproducibility details are needed before drawing stronger conclusions about model capability.